Ajuste fino de VL-JEPA

Metodología e implementación en el Proyecto T7

EL7006 — Redes Neuronales y Teoría de la Información para el Aprendizaje · Universidad de Chile

Fuente: rama **fine-tuning** del repositorio (commit 4eb6acc), documentos `docs/ETAPA2_FINETUNING.md` y `docs/ETAPA2B_REFORMULACION.md`, código en `src/vlmfid/train/` y tablas en `reports/tables/eda_finetuning/`. Documento generado el 2 de noviembre de 2026.

Resumen

VL-JEPA no genera texto token a token: predice el embedding continuo de la descripción objetivo y obtiene texto recuperando la descripción más cercana de un banco. En el proyecto se tomó el checkpoint público *open-vljepa* (MSRVTT, Stage B) y se ajustó con QLoRA sobre Multimodal3DIdent (M3DI), con el mismo presupuesto que los tres VLM autorregresivos: una época sobre las 125 000 imágenes de train y un lote efectivo de 64. El predictor (8 capas de Llama-3.2) recibe LoRA sobre una base NF4, el conector visual se entrena completo y los dos encoders quedan congelados. Como pérdida se usa InfoNCE contra el banco completo de 504 captions canónicos, que es exactamente el conjunto entre el que se elige la respuesta en inferencia. En test, el Índice macro sube de 0,287 a 0,874 y la forma llega a 1,000, pero VL-JEPA queda por debajo de los VLM autorregresivos (≈ 0,999), sobre todo por la posición vertical (0,670).

1. Contexto

El proyecto mide qué tan fiel es la descripción que un modelo multimodal produce de una imagen. M3DI es un conjunto sintético donde cada imagen se generó a partir de atributos conocidos (forma, posición, color del objeto, del foco y del fondo), lo que permite evaluar atributo por atributo y no solo por similitud textual. El proyecto tiene dos etapas:

- **Etapa 1 — inferencia sin ajuste:** cada modelo × 6 prompts sobre las mismas 10 000 imágenes de test.
- **Etapa 2 — ajuste fino:** los cuatro modelos (LLaVA-OV-0.5B, InternVL3-1B, Qwen3.5-0.8B y VL-JEPA) con el mismo presupuesto de cómputo. La comparación entre etapas separa falta de conocimiento de desalineamiento de formato.

VL-JEPA es el punto de contraste central: resuelve la misma tarea en el espacio de embeddings, sin un decodificador autorregresivo.

2. El modelo de partida

Se usa la reimplementación abierta de VL-JEPA (Chen et al., 2025, arXiv:2512.10942) de J. Baek, incluida como submódulo fijado en `external/open-vljepa` (commit f92107f). Los pesos `best.pt` se descargan de `cun-bjy/open-vljepa` en Hugging Face. El envoltorio del proyecto es `src/vlmfid/models/vljepa.py` (`VLJEPADescriber`), que implementa la misma interfaz `describe(images, prompts)` que los demás modelos.

Componente	Qué es	Función
X-Encoder	V-JEPA 2 ViT-L	Imagen → tokens visuales. Procesa video, así que la imagen se repite en 8 fotogramas.
Conector `vis_proj`	Linear 1024 → 2048	Lleva los tokens visuales al ancho del predictor.
Predictor	Últimas 8 capas de Llama-3.2-1B, no causal	(tokens visuales, consulta) → $\hat{s}_Y$ de 1536 dimensiones.

Componente	Qué es	Función
Y-Encoder	EmbeddingGemma-300M + proyección	Texto → s_Y en el mismo espacio de 1536 dimensiones.

Decodificación por recuperación. Para obtener texto legible, el Y-Encoder codifica un banco de descripciones del split de *train* (nunca de test) y se devuelve la más cercana a $\hat{s}_Y$ en coseno. Con el mismo banco se arman prototipos por valor de atributo, que dan una predicción por atributo sin pasar por texto.

Resultado zero-shot (etapa 1). VL-JEPA obtuvo el mejor BLEU/ROUGE de todos los modelos (recupera textos con el formato exacto), pero apenas superó el azar en fidelidad: Índice macro entre 0,285 y 0,291 en los seis prompts, +3,5 puntos sobre el azar, y un CIDEr (0,589) casi igual al de usar la referencia de otra imagen (0,571). Las métricas casi no cambiaban con el prompt: en la práctica ignoraba la consulta.

3. Metodología del ajuste fino

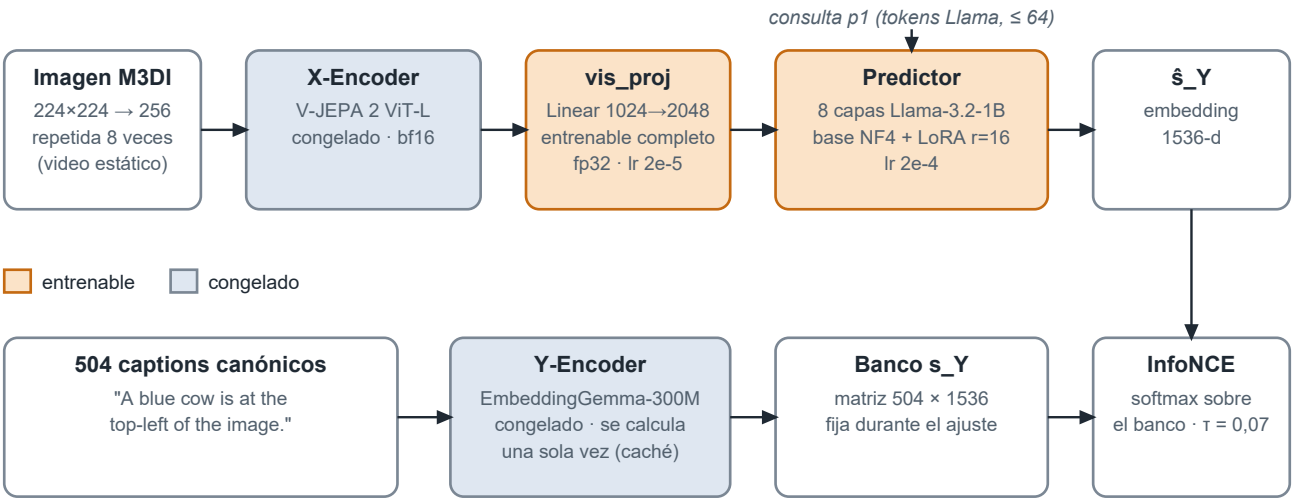


Figura 1. Flujo de entrenamiento de VL-JEPA. Solo el conector y los adaptadores LoRA del predictor reciben gradiente; los textos objetivo se codifican una sola vez con el Y-Encoder congelado.

3.1 Qué se entrena

Componente	Estado	Parámetros entrenables
X-Encoder (V-JEPA 2)	Congelado, bf16, en modo eval	—
Y-Encoder (EmbeddingGemma)	Congelado, en modo eval	—
Predictor (8 capas Llama)	Base NF4 + doble cuantización; LoRA r = 16, α = 32, dropout 0,05 sobre q/k/v/o_proj y gate/up/down_proj	5,64 M
`predictor.vis_proj`	Entrenable completo (copia vía modules_to_save de PEFT), pesos maestros en fp32	2,10 M

Los parámetros entrenables se guardan en fp32 y el forward corre en autocast bf16. Con AdamW y lr 2e-5, una actualización típica del conector queda por debajo de la resolución de bf16: entrenarlo en bf16 lo dejaría prácticamente congelado.

3.2 Texto objetivo: caption canónico

El caption original de M3DI usa 5 plantillas al azar y ~100 nombres de color que no son función del tono (las reglas del EDA lo reproducen solo en ~27 % de los casos). Esa variación es ruido que no depende de la imagen. La reformulación del 23-10-2026 entrena con un caption canónico construido desde los latentes de la imagen:

```
A {color básico} {forma} is at the {vertical}-{horizontal} of the image.
```

- **color**: categoría básica del tono latente (red, orange, yellow, green, cyan, blue, purple, pink); es la misma verdad contra la que se evalúa el color.
- **forma**: teapot, hare, dragon, cow, armadillo, horse, head. **posición**: {top, mid, bottom} × {left, center, right}.
- Hay 7 × 9 × 8 = **504 textos distintos** (frente a 10 936 en el original). El extractor de atributos los recupera con un 100 % de acierto (tests/test_targets.py).
- Consecuencia: la exactitud del *nombre exacto* de color queda en ~0 por construcción, y BLEU/ROUGE/CIDEr frente a la referencia original no son comparables con la etapa 1. El control `target.format=original` mide cuánto costaba aprender el formato arbitrario.

3.3 Función de pérdida

Los VLM autorregresivos usan entropía cruzada por token. VL-JEPA no produce tokens, así que se usa InfoNCE, la misma familia de pérdida contrastiva con que se preentrenó el checkpoint, con la temperatura del preentrenamiento (τ = 0,07). Los negativos son **todos los textos objetivo únicos** (el banco *C* de 504 captions):

$$L = (1/B) \Sigma_i - \log [\exp(\langle \hat{s}_i, s_{c_i} \rangle / \tau) / \Sigma_{j \in C} \exp(\langle \hat{s}_i, s_j \rangle / \tau)]$$

donde $\hat{s}_i$ es la predicción normalizada de la muestra i, s_j el embedding normalizado del texto j, c_i el índice del caption correcto y B el lote efectivo. Es una clasificación softmax entre 504 clases, en una sola dirección (predictor → texto). Tres razones para esta elección:

- **Se optimiza lo que se mide:** en val y test la respuesta se elige entre ese mismo banco por similitud coseno.
- **Sin falsos negativos:** un caption repetido en el lote es la misma clase, así que no hace falta enmascarar duplicados.
- **Acumulación de gradiente:** cada muestra aporta un término independiente del resto del lote, por lo que el lote efectivo se puede partir en micro-lotes (o en mitades ante falta de memoria) sin cambiar la matemática.

Diseño anterior. La primera versión usaba InfoNCE bidireccional con las otras 63 muestras del lote como negativos. En M3DI hay solo ~11 000 captions distintos para 125 000 imágenes, así que dos muestras del mismo lote podían compartir caption; se enmascaraban esos pares para no separar embeddings que debían coincidir. Sigue disponible como `objective.negatives=batch`.

3.4 Presupuesto común y selección

Aspecto	Valor	Comentario
Datos de train	125 000 imágenes, 1 época	1954 pasos; el último lleva las 8 imágenes restantes (drop_last = false)
Lote efectivo	64	Igual en los 4 modelos; VL-JEPA usa micro-lote 64 (sin acumulación)
Optimizador	AdamW, β = (0,9; 0,999), sin weight decay	Dos grupos: LoRA lr 2e-4, conector lr 2e-5
Schedule	Calentamiento lineal 3 % (≈ 59 pasos) + coseno	Clip de gradiente en 1,0
Memoria	Gradient checkpointing por capa del predictor	Como en open-vljepa/scripts/train.py
Prompt	p1_attributes	Igual en los 4 modelos; p4m se truncaba a 64 tokens y los 6 prompts daban lo mismo
Validación	500 imágenes fijas de val, cada 200 pasos y en el paso 0	Paso 0 = adaptador en cero (control de paridad)
Selección	shape_pos = (forma + posición) / 2	Color ±1 estaba saturado y diluía el Índice macro
Test	Las mismas 10 000 imágenes del zero-shot	Solo al final, con adapter_best; nunca durante el ajuste

3.5 Cambios de la reformulación (23-10-2026)

Se hicieron antes de ejecutar cualquier entrenamiento, así que no invalidan resultados.

Aspecto	Antes	Después
Objetivo	Caption original	Caption canónico (original como control)
Prompt de VL-JEPA	p4m (truncado a 64 tokens)	p1, igual que los otros tres
Métrica de selección	Índice macro	shape_pos

Aspecto	Antes	Después
Negativos	Las 63 muestras del lote	Banco completo de textos objetivo
Datos	train_limit para pruebas; drop_last	Train completo siempre; max_steps solo para humo
Experimentos	1 run por modelo	Principal + split composicional + sondas lineales + controles

4. Implementación

VL-JEPA usa el mismo entrenador que los VLM autorregresivos. La clave `objective.kind: infonce` cambia tres piezas: el featurizer, la pérdida y los módulos que quedan congelados. El flujo completo es:

```
scripts/train.py
 ├─ compose_train()          budget.yaml + train/models/vljepa.yaml + línea de comandos
 ├─ train(cfg) -> Trainer.run()                          src/vlmfid/train/loop.py
 │    ├─ _setup()
 │    │    ├─ VLJEPADescriber.load()     checkpoint MSRVTT, NF4 en el predictor
 │    │    ├─ attach_lora()              LoRA + conector vis_proj      (train/lora.py)
 │    │    ├─ JEPAFeaturizer             imagen -> 8 frames + consulta (train/collate.py)
 │    │    └─ _target_table()            Y-Encoder codifica los 504 textos una vez
 │    ├─ AdamW (2 grupos de lr) + calentamiento + coseno
 │    └─ _train_loop() -> _step() -> bank_infonce_loss()     (train/objectives.py)
 │         └─ evaluate() cada 200 pasos -> generate_and_score() -> adapter_best
 └─ evaluate_test() -> scripts/infer.py con adapter_best -> test_metrics.json
```

4.1 Configuración

`train/config.py::compose_train` junta `configs/train/budget.yaml` (común a los 4 modelos) con `configs/train/models/vljepa.yaml`. El archivo de cada modelo solo puede fijar el prompt, el micro-lote, el conector y el objetivo; cualquier otra clave se rechaza, que es la definición operativa de "mismo presupuesto". También iguala `bank.target_format` a `target.format`, para que el banco de recuperación esté en el formato que se entrena.

```
# configs/train/models/vljepa.yaml
prompt: p1_attributes
train:
  micro_batch_size: 64
lora:
  connector_modules: [vis_proj]      # predictor.vis_proj: Linear 1024 -> 2048
objective:
  kind: infonce
  negatives: bank                    # todos los textos objetivo únicos (504)
```

4.2 LoRA sobre un modelo que no es de transformers

`train/lora.py::attach_lora` resuelve dos detalles propios de VL-JEPA:

- VL-JEPA no es un `PreTrainedModel` y la cuantización NF4 se le aplica a mano (`quant.swap_linear_to_nf4`). Se marca `is_loaded_in_4bit = True` para que PEFT elija la capa LoRA compatible con `bnb.Linear4bit`.
- El gradient checkpointing se activa capa por capa sobre `predictor.layers`, envolviendo cada forward con `torch.utils.checkpoint` (no reentrante).

Luego se arma el `LoraConfig` con el regex que expone `VLJEPADescriber.lora_target_modules()` y el conector como `modules_to_save`. La función verifica que no haya parámetros entrenables inesperados, que LoRA se haya aplicado al menos a una capa y que el conector haya quedado entrenable. `set_train_mode` deja `x_encoder` y `y_encoder` en modo eval.

```
# VLJEPADescriber.lora_target_modules()
predictor\.layers\.\d+\..*(q_proj|k_proj|v_proj|o_proj|gate_proj|up_proj|down_proj)$
```

4.3 Datos y tabla de objetivos

`JEPAFeaturizer` (train/collate.py) tokeniza la consulta una sola vez con el tokenizador de Llama y la repite en el lote, registrando si se trunca. Cada imagen pasa por Resize → CenterCrop(256) → normalización de ImageNet y se repite en 8 fotogramas, con forma (B, 8, 3, 256, 256). El objetivo de cada muestra lo produce `data/targets.py::canonical_caption` desde los latentes de la imagen.

Antes de entrenar, `Trainer._target_table` codifica con el Y-Encoder congelado los textos únicos de train y val (504) y los guarda en caché (`data/cache/vljepa_targets_<hash>.pt`). Durante el ajuste el Y-Encoder no vuelve a ejecutarse: los objetivos son una matriz fija de 504 × 1536.

4.4 Paso de entrenamiento

```
# Trainer._jepa_pred + objectives.bank_infonce_loss
vis  = inner.x_encoder(pv)               # congelado: no registra gradiente
pred = inner.predictor(vis, qi, qm)      # LoRA + vis_proj: con gradiente
idx  = [target_index[c] for c in captions]
p, b = normalize(pred), normalize(target_emb)
loss_sum = cross_entropy(p @ b.T / 0.07, idx, reduction="sum")
(loss_sum / B_efectivo).backward()

clip_grad_norm_(entrenables, 1.0); optimizer.step(); scheduler.step()
```

La suma se divide por el tamaño del lote efectivo, de modo que el gradiente acumulado no depende de cómo se partió el lote. Si la GPU se queda sin memoria, el paso se repite con las mismas muestras y el micro-lote a la mitad.

4.5 Validación, selección y reanudación

- **Validación** (`Trainer.evaluate`): en el paso 0 y cada 200 pasos calcula la val_loss con la misma pérdida y corre `generate_and_score`, que hace la inferencia real por recuperación sobre el banco canónico y mide los atributos en 500 imágenes. Si mejora shape_pos, guarda `adapter_best/` de forma atómica. Todo queda en `val_log.jsonl` y `val_predictions/step_*.jsonl`.
- **Reanudación**: cada 100 pasos guarda en `checkpoint/` los parámetros entrenables, el optimizador, el scheduler, el estado de los generadores aleatorios y state.json. El sampler es determinista y retoma en el mismo paso; ante SIGTERM guarda antes de salir.
- **Test** (`scripts/train.py::evaluate_test`): libera el modelo y corre `scripts/infer.py` con `model.adapter_path=adapter_best` y `bank.target_format=canonical`: el mismo camino de inferencia y las mismas 10 000 imágenes que la etapa 1. Escribe `test_metrics.json`.

4.6 Diferencias con los VLM autorregresivos

	VLM (LLaVA-OV, InternVL3, Qwen3.5)	VL-JEPA
Featurizer	GenerativeFeaturizer: plantilla de chat, etiquetas enmascaradas, relleno a la izquierda	JEPAFeaturizer: 8 frames + consulta fija
Pérdida	lm_loss: entropía cruzada por token, normalizada por tokens del lote	bank_infonce_loss: softmax sobre 504 textos, normalizada por muestras
LoRA en	language_model	8 capas Llama del predictor
Conector	Proyector o merger	predictor.vis_proj
Congelados	Encoder visual	x_encoder, y_encoder
Respuesta en val/test	Generación greedy	Vecino más cercano en el banco

Tests. tests/test_train_e2e.py usa un VL-JEPA diminuto (tests/_tiny_vlm.py) que entrena de punta a punta en CPU con ambos tipos de negativos. Comprueba la selección del checkpoint, la recarga del adaptador, que reanudar tras SIGTERM dé exactamente el mismo resultado y que el resultado no dependa del micro-lote.

5. Resultados

Run principal, evaluado en las 10 000 imágenes de test. El checkpoint elegido en val fue el del paso 1800 de 1954.

Atributo	Zero-shot (p1)	Ajustado	Δ	Brecha cerrada
Forma	0,261	**1,000**	+0,739	100 %
Posición x	0,422	0,956	+0,535	92 %
Posición y	0,361	**0,670**	+0,309	48 %
Posición (x, y)	0,158	0,645	+0,486	58 %
Color (categoría exacta)	0,207	0,760	+0,553	70 %
Color ±1	0,442	0,977	+0,535	96 %
Forma + posición	0,210	0,822	+0,613	78 %
Índice macro	0,287	0,874	+0,587	82 %

Todas las diferencias son significativas (McNemar pareado con corrección de Holm, p ≈ 0). "Brecha cerrada" = Δ / (1 − zero-shot).

5.1 Comparación con los otros modelos

Modelo	Forma+pos ZS	Forma+pos FT	Color FT	Macro FT	h GPU	Pico GB	Entrenables (M)
Qwen3.5 Base	0,302	0,9995	0,914	0,9997	0,78	3,5	22,8
InternVL3 1B	0,171	0,9995	0,887	0,9995	1,36	11,4	13,3
LLaVA-OV 0.5B	0,207	0,9990	0,876	0,9992	3,92	10,8	10,6
VL-JEPA	0,210	**0,8222**	0,760	**0,8739**	2,04	16,1	7,7

- VL-JEPA mejora mucho, pero queda último: los tres autorregresivos llegan a ≈ 0,999 en forma y posición.
- **La forma la resuelve por completo (1,000); el cuello de botella es la posición vertical (0,670).** Una hipótesis es que el X-Encoder, o la forma de agregar los tokens visuales en el predictor, pierde la disposición vertical. Las sondas lineales planeadas (stage2b_probe) permitirían confirmarlo.
- Solo el 47 % de sus respuestas coinciden exactamente con el texto objetivo, frente a ≈ 87–91 % en los autorregresivos.
- Es el segundo más caro en tiempo (2,0 h) y el de mayor pico de memoria (16 GB), pese a entrenar menos parámetros.

6. Limitaciones y trabajo pendiente

- Las tablas versionadas solo incluyen el run principal. El split composicional (14 de 63 combinaciones forma × posición retiradas de train), las ablaciones (objetivo original, bf16, semilla) y las sondas lineales están definidos en configs/queues/stage2b_*, pero no hay resultados en el repositorio. El split composicional es, según el diseño, el contraste más directo entre paradigmas.
- Con el objetivo canónico, la exactitud del nombre exacto de color es ~0 por construcción y las métricas de texto no son comparables con la etapa 1; la comparación debe apoyarse en la exactitud por atributo.
- El checkpoint de partida fue entrenado sobre video (MSRVTT); la imagen se trata como video estático de 8 fotogramas, lo que puede limitar la representación espacial.
- La ruta antigua (negatives = batch) no tiene protección contra falta de memoria: el lote de 64 debe caber de una vez. No afecta al run principal, pero sí a la ablación que reproduce el diseño anterior.
- Una sonda lineal baja no prueba que la información no exista: puede estar codificada de forma no lineal o repartida entre tokens.

7. Archivos de referencia

Archivo	Contenido
`src/vlmfid/models/vljepa.py`	Envoltorio de VL-JEPA, banco de recuperación y prototipos
`src/vlmfid/train/loop.py`	Trainer: setup, paso, validación, checkpoints, reanudación
`src/vlmfid/train/lora.py`	attach_lora, grupos de parámetros, modo train/eval
`src/vlmfid/train/collate.py`	JEPAFeaturizer y GenerativeFeaturizer
`src/vlmfid/train/objectives.py`	lm_loss, infonce_loss, bank_infonce_loss
`src/vlmfid/data/targets.py`	Caption canónico y split composicional
`configs/train/budget.yaml`	Presupuesto común a los 4 modelos
`configs/train/models/vljepa.yaml`	Ajustes permitidos de VL-JEPA
`docs/ETAPA2_FINETUNING.md`	Diseño original de la etapa 2
`docs/ETAPA2B_REFORMULACION.md`	Reformulación y plan de experimentos
`reports/tables/eda_finetuning/`	Resultados: zero-shot vs ajustado, costo, formato